

Helix Thready — OpenAPI Structural Validation Report

Table of Contents

1. Summary
2. Tooling availability (what ran, what did not)
3. Errors (2) — must fix
 - 3.1 `#/components/schemas/Error/properties/error/properties/message`
 - 3.2 `#/components/schemas/Asset/properties/renditions/items/properties/url`
4. Warnings (39) — by rule
5. Hand-lint of required OpenAPI 3.1 fields
6. How to reproduce
7. Open items

Rev	Date	Author	Change
1	2026-07-22	swarm (API & SDKs — materials)	Initial validation of <code>../openapi.yaml</code> with <code>@redocly/cli</code> 2.40.0 + a PyYAML/hand-lint of the required OpenAPI 3.1 fields

Table of Contents

1. Summary
2. Tooling availability (what ran, what did not)
3. Errors (2) — must fix
4. Warnings (39) — by rule
5. Hand-lint of required OpenAPI 3.1 fields
6. How to reproduce
7. Open items

1. Summary

`../openapi.yaml` (`openapi: 3.1.0`, `info.version: 1.0.0`; **46 paths, 59 operations, 40 schemas, 1 webhook** — counts confirmed by a PyYAML parse) was validated with `@redocly/cli` 2.40.0 under its built-in `recommended` ruleset [`VERIFIED: tool run`].

Result: **2 errors, 39 warnings, 0 ignored** — validation FAILED.

Both errors are the **same real defect**: a `description` written as an unquoted YAML flow-scalar that contains a comma, so YAML splits the clause after the comma into a **spurious extra schema property**. This is a genuine bug in `openapi.yaml`, not a linter false-positive — a raw PyYAML parse reproduces it independently (§3). It is recorded here and raised as an **open item** because this task may write only inside `materials/`; the fix belongs in `../openapi.yaml` and is a one-line quote per site.

The 39 warnings are style/completeness advisories under the `recommended` ruleset (missing `4XX` on read-only GETs, three unused reusable components, a strict-license note, truncated `...`-style example UUIDs); none blocks codegen and each is triaged in §4.

Provenance: `[VERIFIED: tool run]` = observed output of the named tool in this environment;
`[DEFAULT — adjustable]` = recommended remediation, not yet applied.

2. Tooling availability (what ran, what did not)

Tool	Present?	Used	Notes
<code>@redocly/cli</code> (<code>npx @redocly/cli@latest</code>)	yes — 2.40.0	primary	Ran the full <code>recommended</code> OpenAPI 3.1 ruleset. [VERIFIED: tool run]
<code>buf</code>	yes — 1.71.0 (<code>/home/milos/go/bin/buf</code>)	codegen	Not an OpenAPI validator; drives the proto plane (see <code>codegen/</code>).
PyYAML	yes — 6.0.3	cross-check	Independent parse to confirm the two errors + count paths/ops/schemas.
<code>@stoplight/spectral-cli</code>	not installed		<code>npx</code> install timed out in this sandbox (no cached package); not used. Redocly's <code>recommended</code> ruleset covers the same structural checks, so the gap does not leave the spec unvalidated.
<code>openapi-generator</code> (CLI)	not installed locally	n/a	Available via Docker image <code>openapitools/openapi-generator-cli:v7.9.0</code> or <code>npx</code> ; its <code>--skip-validate-spec=false</code> is a <i>second</i> validator the <code>codegen/Makefile</code> runs at generation time. Not exercised here (no network pull attempted).

Honest tooling gap: only one dedicated OpenAPI validator (`redocly`) actually executed. Spectral and a live `openapi-generator validate` were **not** run (sandbox/network limits). The redocly `recommended` ruleset is a superset of the structural (`struct`) checks that matter for 3.1 conformance, and the PyYAML cross-check independently confirms the two hard errors, so the two blocking defects are **verified**, not merely linter-reported.

3. Errors (2) — must fix

Both are the **struct** rule: `Property <x> is not expected here.` Root cause: a comma inside an **unquoted** flow-mapping `description`, which YAML parses as a key/value separator, so the text after the comma becomes an unexpected property whose value is `null`.

3.1 `#/components/schemas/Error/properties/error/properties/message`

- **Location:** `openapi.yaml:322`
- **Line:** `message: { type: string, description: Human-readable, non-localized-by-default. }`
- **What YAML actually parses** (independently reproduced with PyYAML): `keys -> ['type', 'description', 'non-localized-by-default.']` — the description silently truncates to `"Human-readable"` and a bogus property `non-localized-by-default.` (value `null`) is injected into the schema. `[VERIFIED: PyYAML parse]`
- **Fix** `[DEFAULT – adjustable]`: quote the value — `description: "Human-readable, non-localized-by-default."`

3.2 `#/components/schemas/Asset/properties/renditions/items/properties/url`

- **Location:** `openapi.yaml:633`
- **Line:** `url: { type: string, format: uri, description: Asset-Service-resolved, never a raw path. }`
- **What YAML actually parses:** `keys -> ['type', 'format', 'description', 'never a raw path.']` — description truncates to `"Asset-Service-resolved"` and a bogus property `never a raw path.` (value `null`) is injected. `[VERIFIED: PyYAML parse]`
- **Fix** `[DEFAULT – adjustable]`: quote the value — `description: "Asset-Service-resolved, never a raw path."`

Any inline flow-scalar `description:` containing a comma, colon or `#` must be quoted. A repo-wide guard (`redocly lint` in a pre-commit hook, see `codegen/Makefile openapi-lint`) prevents regressions.

4. Warnings (39) — by rule

Count	Rule	Severity	Assessment
31	<code>operation-4xx-response</code>	advisory	Read-only GET/list operations document only <code>200</code> . Intended, per <code>openapi.yaml</code>'s own preamble : the universal <code>401/429/500</code> (and <code>404</code> where relevant) are declared once in the info/description and “omitted from each operation’s <code>responses</code> for brevity.” Suppress via a <code>.redocly.yaml</code> rule override rather than padding every GET.
3	<code>no-unused-components</code>	advisory	<code>responses.Internal</code> , <code>headers.Sunset</code> , <code>securitySchemes.oauth2</code> are declared but never <code>\$ref</code> -d. They are deliberate contract surface (the universal 500 envelope, RFC 8594 deprecation header, and the OAuth2 linking flow whose only route — <code>/auth/oauth2/authorize</code> — uses a 302, not the scheme). Keep; optionally reference <code>Internal</code> / <code>Sunset</code> from one op each to silence.
2	<code>no-invalid-media-type-examples</code>	advisory	Two example <code>id</code> values are truncated for readability and so fail <code>format: uuid</code> : <code>.../posts/{postId}/process</code> 202 <code>examples.claimed.value.id = "7c9a...job"</code> , and <code>.../events/{...}/sticky</code> 200 <code>examples.lastProgress.value.id = "8f0e...-uuid"</code> . Cosmetic; replace the ellipsis ids with full UUIDs to clear.
1	<code>operation-2xx-response</code>	advisory	<code>GET /auth/oauth2/authorize</code> has only <code>302</code> (by design — it redirects to the provider consent screen). Acceptable for an OAuth start endpoint.
1	<code>info-license-strict</code>	advisory	<code>info.license</code> has <code>name</code> (<code>UNLICENSED — internal...</code>) but no <code>url</code> / <code>identifier</code> . Intentional (not an SPDX id). Add <code>identifier: LicenseRef-Helix-Internal</code> if strict compliance is wanted.
1	<code>operation-operationId</code>	advisory	The <code>eventDelivery</code> webhook operation has no <code>operationId</code> .

Count	Rule	Severity	Assessment
			Webhook ops are not client-invoked; add <code>operationId: onEventDelivery</code> for SDK naming symmetry if desired.

Net: 0 of the 39 warnings indicates a wire-contract defect. The 31 `operation-4xx` warnings are an artifact of the spec’s documented “universal responses declared once” convention colliding with a per-operation linter rule; the recommended action is a scoped rule-severity override, not 31 edits.

5. Hand-lint of required OpenAPI 3.1 fields

Independent of redocly, the mandatory 3.1 structural fields were confirmed present and well-typed via PyYAML `[VERIFIED: PyYAML parse]`:

Required field	Present	Value / note
<code>openapi</code>	✓	<code>3.1.0</code>
<code>info.title</code>	✓	Helix Thready Control API
<code>info.version</code>	✓	<code>1.0.0</code>
<code>paths</code>	✓	46 path items, 59 operations
<code>components.schemas</code>	✓	40 schemas; <code>Error</code> envelope + <code>PageMeta</code> shared
<code>components.securitySchemes</code>	✓	<code>bearerAuth</code> , <code>apiKeyAuth</code> , <code>oauth2</code> , <code>hmacAuth</code>
root <code>security</code>	✓	<code>bearerAuth</code> / <code>apiKeyAuth</code> (per-op overrides with <code>security: []</code>)
<code>servers</code>	✓	prod / staging / dev under <code>/v1</code>
<code>webhooks</code> (3.1)	✓	<code>eventDelivery</code> (outbound, HMAC-signed <code>EventEnvelope</code>)
<code>tags</code>	✓	14 tags; every operation is tagged
<code>\$ref</code> integrity	✓	redocly resolved all refs with no unresolved- <code>\$ref</code> errors

The document is therefore **structurally a valid OpenAPI 3.1 skeleton**; the only conformance blockers are the two `description` mis-parses in §3.

6. How to reproduce

```
cd docs/public/research/mvp/api

# Primary validator (recommended ruleset):
npx --yes @redocly/cli@latest lint openapi.yaml

# Machine-readable, to categorise findings:
npx --yes @redocly/cli@latest lint openapi.yaml --format=json

# Independent confirmation of the two errors (no linter involved):
python3 - <<'PY'
import yaml
d = yaml.safe_load(open('openapi.yaml'))
print('Error.message keys ->',
      list(d['components']['schemas']['Error']['properties']['error']['properties']['message']))
print('Asset.renditions.items.url keys ->',
      list(d['components']['schemas']['Asset']['properties']['renditions']['items']
           ['properties']['url']))
PY
```

Expected: redocly reports `2 errors, 39 warnings`; the PyYAML snippet prints the two spurious keys `non-localized-by-default.` and `never a raw path.`.

7. Open items

- `[OPEN: val-1]` **Fix the two `description` mis-parses in `../openapi.yaml`** (§3.1, §3.2) — one-line quote each. Out of this task's write scope (materials-only); raised for the API owner. Until fixed, `redocly lint` fails and any strict `openapi-generator` run with `--skip-validate-spec=false` will reject the spec.
- `[OPEN: val-2]` Add a scoped `.redocly.yaml` that downgrades `operation-4xx-response` (the spec documents universal responses centrally) and fills the two truncated example UUIDs, so a clean `redocly lint` becomes a real pre-tag gate (wired in `codegen/Makefile: openapi-lint`).
- `[OPEN: val-3]` **Tooling gap:** neither Spectral nor a live `openapi-generator validate` ran here (sandbox/network). Re-run both in an environment with network to get a second and third independent structural opinion before publishing the SDKs.

Made with love ♥ by Helix Development.